

臺北城市儀表板（Taipei City Dashboard）完整技術文件重點摘要

臺北城市儀表板是由臺北市政府資訊局「臺北大數據中心（TUIC）」開發的 [Citydashboard](#) 開源資料視覺化平台，[GitHub](#) 採用前端（FE）、後端（BE）、資料端（DE）三層架構，整合統計資料與地理空間資料 [GitHub](#) [GitHub](#) 於模組化的儀表板組件中，[Gov](#) [Gov](#) 並以 Docker、Kubernetes 支撐容器化部署。專案從 2023 年的內部工具（v1.0）演化為公開開源平台（v2.0+），[GitHub](#) [GitHub](#) 目前最新版本為 **v3.1.0**（2025 年 8 月），[github](#) 累計 **1,660+ 次提交**、**27 位貢獻者**、**591 個 Fork**。[github](#) 其核心理念為「Our dashboard, made yours」——任何組織皆可下載程式碼、接入自有資料，快速建立類似的城市資料視覺化平台。[GitHub](#) [GitHub](#)

一、整體架構與技術棧總覽

Monorepo 頂層結構

```
Taipei-City-Dashboard/
├── Taipei-City-Dashboard-FE/    # 前端 (Vue 3)
├── Taipei-City-Dashboard-BE/    # 後端 (Go/Gin)
├── Taipei-City-Dashboard-DE/    # 資料端 (Python/Airflow)
├── db-sample-data/              # 資料庫範例資料 (含 SQL schema)
├── docker/                      # Docker 部署配置
│   ├── nginx/ssl/              # Nginx 反向代理 + SSL
│   └── db-data/                 # PostgreSQL 資料持久化
├── helm-chart/                  # Kubernetes Helm 部署圖表
└── .github/                     # CI/CD、Issue 模板、貢獻指南
```

程式語言比例（整體倉庫）

語言	比例	對應模組
Python	38.6%	資料端（Airflow DAGs、ETL）
Vue	34.1%	前端 SPA
JavaScript	15.9%	前端輔助
Go	9.2%	後端 REST API
PLpgSQL	~23%*	資料庫 Schema（含 stored procedures）
TypeScript	0.7%	前端型別

*PLpgSQL 比例來自 Fork 倉庫語言統計，確認 PostgreSQL 內含大量預存程序。[github](#)

部署架構

前端經 **Vite** 建置後由 **Nginx** 提供靜態檔案服務並擔任反向代理，所有 API 請求轉送至 **Go/Gin** 後端伺服器，後端連接 **PostgreSQL** 資料庫與 **Redis** 快取。資料端以 **Apache Airflow** 排程 DAG，定時從臺北市開放資料平台與各局處 API 擷取資料，經清洗轉換後寫入 PostgreSQL。整體以 **Docker Compose** 容器化部署，生產環境可使用 **Helm Chart** 部署至 Kubernetes 叢集。

二、前端（FE）技術文件重點

技術棧

前端採用 **Vue 3**（Composition API + `<script setup>` 語法），以 **Vite** 為建置工具，**Pinia** 管理全域狀態，**Vue Router** 處理頁面路由。地圖視覺化使用 **Mapbox GL JS**，圖表渲染使用 **ApexCharts**（vue3-apexcharts）。整體支援**深色/淺色模式**切換、**GitHub**響應式設計，並內建操作教學引導功能。**GitHub**

技術	用途
Vue 3 (Composition API)	UI 框架
Vite	建置與開發伺服器（HMR 快速熱重載）
Pinia	全域狀態管理
Vue Router	SPA 路由
Mapbox GL JS	地理空間資料地圖視覺化
ApexCharts	統計圖表渲染
Axios	HTTP 請求（與後端 API 通訊）

環境設定透過 `.env` 系統管理，包含 `.env.development`、`.env.production`、`.env.test` 三套環境配置，**GitHub**建置產出存放於 `dist/` 目錄。**GitHub**

專案結構

依 **Vue 3** 標準實踐與文件頁面名稱推斷，前端核心目錄結構如下：

```
Taipei-City-Dashboard-FE/
├─ src/
│   ├─ components/           # 可複用 Vue 組件
│   │   ├─ charts/           # 圖表組件 (BarChart, DonutChart, ColumnChart 等)
│   │   ├─ map/               # 地圖相關組件 (MapChart, MapLegend 等)
│   │   └─ utilities/         # 通用 UI 組件
│   ├─ views/                 # 頁面視圖 (Dashboard, Admin, Map 等)
│   ├─ store/                 # Pinia 狀態管理
│   │   ├─ mapStore           # 地圖狀態 (圖層、視角、篩選)
│   │   ├─ contentStore       # 組件內容狀態
│   │   ├─ dialogStore        # 對話框/模態框狀態
│   │   └─ authStore          # 認證狀態
│   ├─ router/                # Vue Router 路由定義
│   ├─ assets/                # 靜態資源 (圖片、樣式)
│   └─ utils/                 # 工具函式
├─ .env.development           # 開發環境變數
├─ .env.production            # 生產環境變數
└─ package.json               # NPM 依賴定義
```

儀表板組件系統

儀表板的核心設計為**模組化資料組件** (Modularized Data Components)，[Gov](#)每個組件是獨立的資料視覺化單元，[GitHub](#)透過統一的配置格式進行管理。[Gov](#)官方文件中明確區分了以下幾個配置維度：

chart_config (圖表配置)：定義統計圖表的類型與顯示參數，支援的圖表類型包括 **BarChart** (長條圖)、**DonutChart** (甜甜圈圖)、**ColumnChart** (柱狀圖)、**BarPercentChart** (百分比長條圖)、**GuageChart** (儀表圖)、**TreemapChart** (矩形樹狀圖)、**TimelineSeparateChart** (時間軸圖)、**RadarChart** (雷達圖) 等。

map_config (地圖配置)：定義 Mapbox GL JS 的地圖圖層設定，包含 **type** (圖層類型：fill、line、circle、symbol、fill-extrusion、heatmap 等)、**paint** (繪製樣式：顏色、透明度、寬度等)、**source** (資料來源配置)。

map_filter (地圖篩選)：控制地圖資料的篩選條件，允許使用者互動式過濾地圖上的資料點。

文件站中有專門頁面 [/front-end/chart-data](#) 和 [/front-end/custom-maps](#) 分別詳述圖表資料格式與自訂地圖的配置方式。

新增 Dashboard 組件的流程

基於 [GitHub Issue #149](#) 和文件結構，新增組件的流程包括：在管理介面 (Admin Edit Component Dialog) 中配置組件的 **chart_config**、**map_config** 和資料來源設定，系統支援即時預覽。開發者可按照設計規範開發全新的圖表類型 (New Chart Type) 或地圖類型 (New Map Type)，[GitHub Issue](#) 模板明確提供了這兩種擴展類別的提交管道。[GitHub](#)

開發流程

1. **Clone** 倉庫（develop 分支）並安裝 Node.js 依賴
2. 配置 `.env` 環境變數（含 Mapbox API Key，需在 Mapbox 後台將 localhost 加入白名單）
3. 透過 **Docker Compose** 一鍵啟動前後端及資料庫服務
4. 執行 `npm run dev` 啟動 Vite 開發伺服器（支援 HMR）
5. 提交前遵循 ESLint 與 Prettier 程式碼規範
6. Fork → Feature Branch → Pull Request 工作流，PR 需以英文撰寫（GitHub）

三、後端（BE）技術文件重點

技術棧

後端以 **Go** 語言撰寫，（github）使用 **Gin** 高效能 HTTP 框架，搭配 **PostgreSQL** 關聯式資料庫及 **Redis** 記憶體快取。編譯後的二進位檔案名稱為 `TaipeiCityDashboardBE`。（GitHub）專案使用 `go.work` 多模組工作區管理。

技術	用途
Go (Golang)	後端主語言
Gin	HTTP Web 框架（REST API）
PostgreSQL	主要關聯式資料庫
Redis	Session/Token 快取管理
GORM	Go ORM（資料庫操作抽象層）
JWT	JSON Web Token 認證
臺北通 (Taipei Pass)	OAuth 第三方登入整合

專案結構

```
Taipei-City-Dashboard-BE/
├── TaipeiCityDashboardBE/
│   ├── cmd/                # 應用程式進入點 (main.go)
│   ├── internal/
│   │   ├── handlers/       # HTTP 請求處理函式
│   │   ├── models/         # 資料模型 (對應 PostgreSQL 資料表)
│   │   ├── routes/         # 路由定義 (Gin Router Group)
│   │   ├── middleware/     # 中介層 (認證、CORS、日誌等)
│   │   └── config/         # 配置管理
```

```
└─ go.mod
└─ go.sum
```

```
# Go 模組依賴定義
# 依賴校驗碼
```

API 端點格式

後端採用 **RESTful** 設計風格，[\(GitHub\)](#)API 路徑前綴為 `/api/v1/`，使用 Gin 的 `router.Group()` 組織路由群組並套用中介層。根據文件頁面結構，API 分為以下幾組：

API 群組	文件頁面路徑	功能說明
Dashboard APIs	<code>/back-end/dashboard-apis</code>	儀表板 CRUD、公開/個人儀表板清單
Component Data APIs	<code>/back-end/component-data-apis</code>	組件資料查詢 (<code>chart_data</code> , <code>map_data</code>)
Component Config APIs	<code>/back-end/component-config-apis</code>	組件配置管理
Issue APIs	<code>/back-end/issue-apis</code>	問題回報建立、查詢、狀態管理
Contributor APIs	<code>/back-end/contributor-apis</code>	貢獻者資訊查詢
Authentication APIs	<code>/back-end/authentication-apis</code>	登入、Token 管理、登出

所有 API 回應格式為 JSON，使用 Gin 的 `c.JSON()` 方法回傳標準化的 HTTP 狀態碼與資料結構。

資料庫 Schema

文件中有專門的 [\(Dashboards DB\)](#) 頁面 (`/back-end/dashboards-db`)，[\(db-sample-data/\)](#) 目錄提供完整的 SQL schema 和種子資料。資料庫使用 **PostgreSQL** 搭配 **PLpgSQL** 預存程序，核心資料表包括：

- **dashboards**：儀表板主表 (ID、名稱、描述、建立者、權限、排序等)
- **components**：組件主表 (ID、名稱、`chart_config`、`map_config`、資料來源、更新頻率、歷史資料配置等)
- **dashboard_components**：儀表板與組件的多對多關聯表
- **users**：使用者資料表
- **issues**：問題回報表
- **contributors**：貢獻者資料表

認證機制

認證採用 **臺北通 (Taipei Pass / TownPass)** **OAuth 2.0** 第三方登入，登入成功後由後端簽發 **JWT Access Token** 與 **Refresh Token**。Access Token 以 Bearer Token 方式附加於 API 請求標頭，[\(API7.ai\)](#)Token 生命週期透過 Redis 管理，支援刷新與撤銷機制。

中介層（Middleware）架構包含：**認證中介層**（驗證 JWT 有效性）、**CORS 中介層**（處理跨域請求）、**** (GitHub) 權限中介層 ****（角色式存取控制）、**Logger 中介層**（請求日誌）、**Recovery 中介層**（panic 恢復）。

開發流程

- 1. 安裝 Go、PostgreSQL、Redis
- 2. Clone 倉庫並設置 `.env` 環境變數
- 3. 匯入 `db-sample-data/` 中的範例資料至 PostgreSQL
- 4. 執行 `go run` 或使用 `air` 進行熱重載開發
- 5. 遵循 `/back-end/code-style` 定義的程式碼風格規範

四、資料端（DE）技術文件重點

技術棧

資料端以 **Python** 撰寫，使用 **Apache Airflow** 作為資料管線排程與編排工具，資料儲存於 **PostgreSQL**（可能含 PostGIS 空間資料擴充），整體透過 **Docker** 容器化部署。

技術	用途
Python	資料處理主語言
Apache Airflow	DAG 排程、管線編排、監控
PostgreSQL (+ PostGIS)	資料儲存（含空間資料）
Docker / Docker Compose	容器化部署（開發/生產分離）
Git Sync	DAG 程式碼自動同步

目錄結構

```
Taipei-City-Dashboard-DE/
├─ dags/                # Airflow DAG 定義檔案
│   ├── settings/      # DAG 通用設定
│   ├── utils/         # 工具函式模組
│   └─ [各個 DAG .py 檔] # 個別資料管線定義
├─ docker/
│   ├── develop/
│   │   ├── .env        # 開發環境變數
│   │   └─ gitsync.sh   # Git 同步腳本（自動拉取最新 DAG）
```

```
|   └─ prod/
|       └─ .env                # 生產環境變數
|       └─ gitsync.sh          # 生產環境 Git 同步
└─ local_env/
    └─ volume_data/           # 本地開發 Airflow 卷資料
    └─ .env                   # 本地環境變數
└─ logs/                      # Airflow 任務日誌
└─ data/                      # 資料暫存目錄
```

DAG 結構與 CommonDag 用法

Airflow 的核心概念為 **DAG**（**Directed Acyclic Graph**，有向無環圖），[Astronomer](#) [Qubole](#) 每個 DAG 代表一條完整的 ETL 資料管線。[Apache Airflow +2](#) 本專案設計了 **CommonDag** 通用 DAG 範本，作為標準化 ETL 管線的抽象工廠，開發者只需提供配置參數即可快速建立新的資料管線，大幅減少重複程式碼。

CommonDag 的典型流程為：從資料來源擷取（**Extract**）→ 轉換清理（**Transform**）→ 寫入資料庫（**Load**）→ 觸發後續更新任務。[Astronomer](#) 文件中的 [/data-end/dag-code](#) 頁面詳述 DAG 程式碼的撰寫規範與結構，[/data-end/dag-test](#) 頁面說明 DAG 測試方法。

ETL 流程與 Utils 函式

資料端遵循標準 **ETL**（**Extract → Transform → Load**）三階段架構，[Apache](#) [DASCA](#) 並提供一組通用工具函式：

函式名稱	所屬階段	功能說明
extract_stage	Extract	從臺北市開放資料平台、各局處 API、外部資料來源下載原始資料
transform_time	Transform	標準化各種時間格式（民國年↔西元年、不同日期格式統一等）
transform_geometry	Transform	空間資料轉換（經緯度座標轉換、區域多邊形處理、CRS 轉換，搭配 PostGIS）
load_stage	Load	將清理後的資料批次寫入 PostgreSQL 資料庫目標資料表

文件中有專門的 [/data-end/utils-load](#) 頁面說明載入階段的工具函式，以及 [/data-end/utils-overview](#)（通用函式概覽）提供所有工具函式的總覽。

job_config.json 格式說明

[job_config.json](#) 是外部化的 DAG 作業配置檔，採用 JSON 格式定義每條資料管線的參數。其典型欄位包括：

- dag_id**：DAG 唯一識別碼
- 資料來源配置**：API URL、資料格式、認證資訊

- **排程設定**：Cron 表達式定義執行頻率（如每日、每小時）
- **目標資料表**：寫入的 PostgreSQL 資料表名稱
- **轉換參數**：時間欄位名稱、地理欄位名稱、資料清理規則
- **更新模式**：全量覆寫或增量更新

此設計將管線邏輯與配置分離，實現「一次撰寫 CommonDag，多次配置 job_config」的高效開發模式。

資料庫 table 命名規則

根據 `db-sample-data/` 目錄的存在以及專案設計原則，資料庫採用有系統的命名規則組織資料表。組件資料表通常依據資料來源與主題命名，與 `chart_data`、`history_data`、`map_data` 等資料格式對應。文件中的 `/data-end/pipeline` 頁面涵蓋完整的資料管線架構與命名慣例。

開發流程

1. **本地開發**：使用 `local_env/` 配置啟動本地 Airflow 環境（Docker Compose）
2. **DAG 開發**：在 `dags/` 目錄下建立新的 DAG .py 檔，引用 CommonDag 與 utils 工具函式
3. **配置設定**：編寫對應的 `job_config.json` 配置檔
4. **測試驗證**：遵循 `/data-end/dag-test` 的測試流程驗證 DAG 正確性
5. **部署**：透過 `gitsync.sh` 自動同步最新 DAG 至開發/生產環境的 Airflow 容器
6. **監控**：透過 Airflow Web UI 監控 DAG 執行狀態、查看任務日誌 `DASCA`

五、共通開發與維運重點

Docker 部署架構

專案使用 **Docker Compose** 一鍵啟動所有服務，核心容器包括：

- **dashboard-fe**：前端 Vue.js 應用（Nginx 提供服務）
- **dashboard-be**：後端 Go/Gin API 伺服器
- **postgres**：PostgreSQL 資料庫（含 PostGIS 擴充）
- **redis**：Redis 快取（Token 管理）
- **nginx**：反向代理（SSL 終止、請求轉發） `GitHub`
- **airflow**：Airflow Scheduler + Worker（DE 管線排程）

Kubernetes 生產部署

`helm-chart/` 目錄提供 Helm 部署圖表，支援雲端 Kubernetes 環境的容器編排，適用於大規模生產部署。

貢獻流程

專案遵循標準開源協作模式：先開 Issue 討論 → 獲批後 Fork 倉庫 → 在 Feature Branch 開發 → 提交 Pull Request。
(GitHub) (GitHub)所有 Issue 和 PR 須以英文撰寫，(GitHub)文件貢獻須同時包含英文與繁體中文版本。
(github +2)GitHub 組織已從 (tpe-doit) 更名為 (taipei-doit)，(GitHub)舊連結自動重導向。

開源策略

據 iThome 報導 (2024 年 1 月)，專案採取類似 Chrome/Chromium 的「大教堂模式」——由臺北市政府維護官方版本 (Taipei City Dashboard)，同時釋出核心程式碼 (City Dashboard) 供外部組織使用。
(iThome +2)臺北市定期舉辦「程式設計節」(Codefest) 黑客松，邀請社群開發新組件，
(GitHub) (GitHub)歷屆參賽作品包括城市韌性等高線圖組件、色盲無障礙框架等。
(iThome)2024 年更進一步開源了後端與資料端程式碼，完成前、後、資料三端的全面開源。
(iThome +2)

六、關鍵資源連結

資源	網址
正式網站	https://citydashboard.taipei
完整技術文件	https://citydashboard.taipei/documentation/
主程式碼庫	https://github.com/taipei-doit/Taipei-City-Dashboard
文件原始碼庫	https://github.com/taipei-doit/Taipei-City-Dashboard-Documentation
TUIC 官方網站	https://tuic.gov.taipei/en
專案設置教學影片	https://www.youtube.com/watch?v=6Qrf_6e9rGY

研究說明：本報告綜合 GitHub 倉庫結構分析 (.gitignore、語言統計、目錄結構、Release 紀錄)、搜尋引擎索引的文件頁面標題與 URL、iThome 新聞報導、SITCON 黑客松參賽紀錄、臺北市政府新聞稿等多方來源編撰。由於文件網站為 Vue SPA 且 robots.txt 限制自動化存取，部分具體配置格式的細節為基於確認的技術棧與軟體工程最佳實踐的合理推斷。建議直接瀏覽 <https://citydashboard.taipei/documentation/> 或 clone 文件倉庫取得最精確的內容。